

GRAPH

PROBLEM 3

QUESTION:

DS

Session

Graph

Question Information

Level 1

Challenge 83

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n .

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$.

After this, there are m lines describing the teleporters. Each line has two integers a and b : there is a teleporter from the room a to room b .

There are no two teleporters whose starting and ending rooms are the same.

Output

First print an integer k : the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6

EXPECTED OUTPUT

2
4
1 3 5 6
3
1 2 6

Test Case 2

INPUT (STDIN)

6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6

EXPECTED OUTPUT

1
4
1 2 3 6

Mandatory Test Cases

Test Case 1

KEYWORD

int bfs(int n,int s,int t)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

920

Test Case 3

NLOC

110

SOURCE CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_ROOMS 505
#define MAX_CHANNELS 2005

typedef struct NetworkArc {
    int target, capacity, residual_flow, inverse_idx;
} NetworkArc;

NetworkArc arcs[2 * MAX_CHANNELS];
int arc_head[MAX_ROOMS], next_arc[2 * MAX_CHANNELS], arc_counter = 0;

void insert_network_arc(int u, int v, int cap) {
    arcs[arc_counter] = (NetworkArc){v, cap, 0, arc_counter + 1};
    next_arc[arc_counter] = arc_head[u];
    arc_head[u] = arc_counter++;

    arcs[arc_counter] = (NetworkArc){u, 0, 0, arc_counter - 1};
    next_arc[arc_counter] = arc_head[v];
    arc_head[v] = arc_counter++;
}

int backtrack_arc[MAX_ROOMS];

bool search_augmenting_path(int total_nodes, int src, int sink) {
    for (int i = 1; i <= total_nodes; i++) backtrack_arc[i] = -1;
    int bfs_queue[MAX_ROOMS], q_front = 0, q_back = 0;
    bfs_queue[q_back++] = src;
    backtrack_arc[src] = -2;

    while (q_front < q_back) {
        int cur = bfs_queue[q_front++];
        if (cur == sink) return true;
        for (int a = arc_head[cur]; a != -1; a = next_arc[a]) {
            int nxt = arcs[a].target;
            if (backtrack_arc[nxt] == -1 && arcs[a].capacity > arcs[a].residual_flow) {
                backtrack_arc[nxt] = a;
                bfs_queue[q_back++] = nxt;
            }
        }
    }
    return false;
}

int main() {
    int rooms, passages;
    if (scanf("%d %d", &rooms, &passages) != 2) return 0;

    for (int i = 1; i <= rooms; i++) arc_head[i] = -1;
    for (int i = 0; i < passages; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        insert_network_arc(u, v, 1);
    }

    int optimal_days = 0;
    while (search_augmenting_path(rooms, 1, rooms)) {
        for (int v = rooms; v != 1; ) {
            int a = backtrack_arc[v];
            arcs[a].residual_flow += 1;
            arcs[arcs[a].inverse_idx].residual_flow -= 1;
            v = arcs[arcs[a].inverse_idx].target;
        }
        optimal_days++;
    }

    printf("%d\n", optimal_days);
    for (int d = 0; d < optimal_days; d++) {
        int route[MAX_ROOMS], length = 0;
        int current_pos = 1;
        route[length++] = current_pos;
        while (current_pos != rooms) {
            for (int a = arc_head[current_pos]; a != -1; a = next_arc[a]) {
                if (arcs[a].residual_flow == 1) {
                    arcs[a].residual_flow = 0;
                    current_pos = arcs[a].target;
                    route[length++] = current_pos;
                    break;
                }
            }
        }
        printf("%d", length);
        for (int i = 0; i < length; i++) printf(" %d", route[i]);
        printf("\n");
    }
    return 0;
}

```

OUTPUT:

```
6 7
1 2
1 3
3 4
3 5
4 6
5 6
2 6
2
3 1 2 6
4 1 3 4 6
```

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
1
4
1 2 3 6
```